Transportation Research Part E 201 (2025) 104220

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

Two-stage stochastic fleet and battery sizing with routing optimization for sidewalk delivery robots

Yuchen Du [a], Hai Yang [b], Joseph Y.J. Chow [b], Tho V. Le [a,*]

[a] School of Engineering Technology, Purdue University, West Lafayette, IN, 47907, USA
[b] C2SMARTER Center, Department of Civil & Urban Engineering, New York University Tandon School of Engineering, Brooklyn, NY, USA

ARTICLE INFO

Keywords:
Sidewalk delivery robots
Food delivery
E-commerce
Routing problem
Pickup-delivery problem with time windows
Continuous approximation

ABSTRACT

The rapidly growing online food delivery (OFD) market presents substantial logistical challenges for last-mile delivery operations. Sidewalk delivery robots (SDRs) have emerged as a promising alternative to on-demand workers, as these compact, box-sized robots efficiently deliver food or groceries over short distances via sidewalks. We propose a two-stage stochastic optimization model for a single-depot SDR system with integrated battery-swapping operations. In the first stage, a continuous approximation (CA) method determines the optimal fleet size and the required number of additional swappable batteries. The second-stage solutions are critical to facilitate the first-stage method. These involve solving a routing problem that incorporates battery-swapping decisions and penalties for late arrivals. To address this, we develop a customized heuristic based on adaptive large neighborhood search (ALNS) to generate high-quality solutions for the second stage. The fitted CA model integrates key factors, including time windows, battery swapping, and pickup-delivery orders. Numerical examples highlight the proposed approach's efficiency in reducing computational time while maintaining solution accuracy. A case study and sensitivity analysis conducted on Purdue University's campus illustrate the practical impacts of fleet size and the number of swappable batteries.

1. Introduction

The food delivery industry has experienced remarkable growth in recent years, driven by increasing consumer demand for convenience and speed. Major players such as Uber Eats (Uber Eats, 2024), Chowbus (Chowbus, 2024), and DoorDash (DoorDash, 2024) have become household names. However, the industry faces persistent challenges, including driver shortages, high operational costs, inefficient routing, and environmental concerns. One promising alternative is the use of Sidewalk Delivery Robots (SDRs), which offer a cost-effective and sustainable solution for last-mile delivery.

SDRs are pedestrian-sized, electrically powered robots designed to navigate sidewalks at a slow pace, making them ideal for dense urban areas with restricted road access (Jennings and Figliozzi, 2019; Garus et al., 2024). Unlike Autonomous Delivery Robots (ADRs), which have larger payload capacities and longer ranges, SDRs are optimized for compact, short-distance deliveries. Successful implementations by companies such as Starship Technologies (Starship Technologies, 2024) and Kiwibot (Kiwibot, 2024) have demonstrated their potential to improve delivery efficiency while reducing operational costs and environmental impact.

The rapid growth of e-commerce has intensified last-mile delivery inefficiencies, which often constitute the most costly and time-consuming segment of the supply chain (Alverhed et al., 2024; Garus et al., 2024). SDRs address these challenges by reducing delivery costs—studies indicate cost reductions of up to 68% —enhancing delivery speed, alleviating urban congestion, and lowering

* Corresponding author.
E-mail addresses: du313@purdue.edu (Y. Du), hy1236@nyu.edu (H. Yang), joseph.chow@nyu.edu (J.Y.J. Chow), thovle@purdue.edu (T.V. Le).

https://doi.org/10.1016/j.tre.2025.104220
Received 23 December 2024; Received in revised form 21 May 2025; Accepted 23 May 2025
Available online 11 June 2025

CO_2 emissions (Alverhed et al., 2024; Jennings and Figliozzi, 2019). Their adaptability to pedestrian zones and traffic-restricted areas further enhances their suitability for urban logistics (Garus et al., 2024).

Despite promising advances in the use of SDRs for food delivery, several logistical challenges must be addressed to fully unlock their potential. A key challenge is optimizing delivery routes to ensure timely and efficient service while meeting constraints such as time windows, robot capacities, and battery consumption. For strategic resource planning, this can be modeled as a Pickup and Delivery Problem with Time Windows (PDPTW), a well-known combinatorial optimization problem. While the actual SDR operation involves online optimization, a static model can provide insights on resources like fleet sizing or allocation of batteries. The PDPTW determines the optimal set of routes for a fleet of vehicles to satisfy customer orders, each with specific pickup and delivery locations and time constraints. For SDRs, additional considerations include relaxing time constraints with penalties for late orders and incorporating charging behavior due to limited battery capacity.

The PDPTW is NP-hard, as demonstrated in the work of Ropke et al. (2007). Few studies have addressed this in a stochastic setting, focusing on resource allocation for deploying a sidewalk robot online food delivery (SROFD) system. In this study, we propose a two-stage stochastic problem. The decision variables include the optimal robot fleet size and the number of additional swappable batteries needed. Order demand levels, locations, and service time windows are stochastic.

We propose a Mixed Integer Programming (MIP) model for the two-stage problem. However, due to the NP-hard nature of the PDPTW, obtaining exact solutions for large-scale instances is generally infeasible. Therefore, we propose a approximation-based heuristic to solve the problem. We solve a sample of scenarios and fit an approximation model extended from Yang et al. (2025). The approximation method is used to find the solution that is likely to minimize the expected system cost. The solution, along with a few other selected solutions that can produce a similar level of expected system cost, are further evaluated. Specifically, multiple scenarios are simulated in the second stage, and the expected system cost is estimated by averaging costs across all scenarios.

The second stage problem is solved using an Adaptive Large Neighborhood Search (ALNS) algorithm tailored to SDRs. ALNS dynamically adjusts the weights of removal and insertion heuristics based on their performance and employs a simulated annealing acceptance criterion to escape local optima. We extend the ALNS framework by incorporating a strategy for recharging SDRs through battery swapping, ensuring continuous operation and enhancing delivery system efficiency. Combining the first stage approximation with the second stage ALNS provides a high-quality solution for resource allocation in an SROFD system.

To validate our proposed method, we conduct extensive experiments. Due to limited real-world applications, we developed a custom instance generator tailored to SDR delivery scenarios. This generator creates datasets within confined areas that accurately reflect the operational and environment constraints of SDRs. Using this custom-generated data, we tested our ALNS algorithm, and the results demonstrate its effectiveness in optimizing SDR routes and highlight the feasibility of deploying SDRs for food delivery services.

This work makes several key contributions to the field of PDPTW for food delivery services. The primary contribution lies in the development of a novel two-stage stochastic optimization framework tailored to sidewalk delivery robot (SDR) operations under uncertainty. While our formulation follows the classical EV-PDPTW structure, we incorporate SDR-specific constraints and parameter settings (e.g., short-range battery limits, soft time windows, depot-based battery swapping), making the framework directly applicable to real-world SDR service design.

The detailed contributions are as follows:

1. We investigate the novel use of SDRs in the PDPTW context and introduce a battery swapping strategy for recharging.
2. We propose a two-stage stochastic model to determine the optimal SDR fleet size and the number of swappable batteries needed to meet dynamic demands in a given area.
3. We develop an approximation-based heuristic for the two-stage model. Simulated scenarios from stage 2 are generated and solved. A continuous approximation (CA) based analytical model is fitted to those solutions as a cost function for the first stage problem.
4. We develop a customized ALNS heuristic to solve each of the simulated scenarios.
5. We validate the effectiveness of our approach using the Purdue University campus as a case study, providing further insights into the feasibility of deploying SDRs for campus delivery services.

The remainder of the paper is organized as follows: Section 2 reviews the related studies. Section 3 introduces the problem setting, and mathematical formulations using mixed integer programming. Section 4 presents our proposed solution method. Section 5 presents the conducted experiments that validate the proposed solution methods. Section 6 shows our case study on Purdue Campus. Section 7 presents the results of the sensitivity analysis for some important parameters. Section 8 concludes our work and shows the potential developments in the future.

2. Literature review

PDPTW is a complex variant of the well-studied Traveling Salesman Problem (TSP). Time constraints, or time windows, specify that each customer must be visited within a predefined time interval. This means that the vehicle must arrive at a customer's location no earlier than the start of the time window and no later than its end (Solomon, 1987). These constraints add complexity, as the solution must not only minimize the total travel distance but also ensure that all service times fall within the specified windows. The PDPTW introduces the additional complexity of pairing pickup and delivery locations for each request, meaning that each vehicle must not only adhere to time windows but also ensure that pickups are completed before corresponding deliveries. The formal definition and initial solutions for PDPTW were presented in Savelsbergh and Sol (1995).

The complexity of PDPTW has led to the development of various solution methods. These methods, including exact algorithms (Aziez et al., 2020; Baldacci et al., 2011) and heuristics, aim to find high-quality solutions efficiently within reasonable computation times. There are many heuristic and meta-heuristic methods, including Large Neighborhood Search (LNS) (Shaw, 1997), Adaptive LNS (ALNS) (Ropke and Pisinger, 2006), Tabu search (TS) (Nanry and Barnes, 2000), granular TS (GTS) (Goeke, 2019), particle swarm optimization (Ai and Kachitvichyanukul, 2009), and simulated annealing (Wang et al., 2015).

Recent research has introduced additional complexities into PDPTW to reflect real-world scenarios better. One such extension is incorporating charging behavior for the new technologies in transportation, such as electric vehicles (EV), drones, and SDRs. The solution for this new PDPTW should ensure these technologies can recharge their batteries when necessary, which means incorporating charging stops into the routes.

Schneider et al. (2014) introduced the concept of electric vehicles with limited battery capacity into the Vehicle Routing Problem with Time Windows (VRPTW), extending it into the Electric VRPTW (E-VRPTW). Since then, there has been a wealth of research on E-VRPTW, including works by Hiermann et al. (2016), Keskin and Çatay (2016) and Erdelić and Carić (2022). In particular, a few studies have considered battery swapping (Yang and Sun, 2015; Jie et al., 2019; Raeesi and Zografos, 2020). However, few studies address the PDPTW considering charging needs. Notable ones include Goeke (2019) and Grandinetti et al. (2016).

Beyond these works, recent research has explored more advanced EV-based optimization challenges, including charging synchronization, dynamic fleet management, and battery-aware routing. Ma et al. (2024) studied electric feeder services, introducing charging synchronization constraints to optimize vehicle charging schedules and minimize waiting times at stations. Similarly, Al-Kanj et al. (2020) developed an approximate dynamic programming (ADP) framework to dynamically manage fleet dispatching, repositioning, and charging decisions for an autonomous EV ride-hailing system. From a routing perspective, Bongiovanni et al. (2019) proposed the Electric Autonomous Dial-a-Ride Problem (e-ADARP), which incorporates battery management, partial recharging, and vehicle-to-depot assignments to improve the operational efficiency of autonomous EV fleets.

While EVs in routing problems face challenges such as range limitations, long charging times, infrastructure dependency, and dynamic traffic conditions (Erdelić and Carić, 2019), SDRs operate in smaller, controlled environments like sidewalks, where energy consumption is more predictable, and battery swapping at depots eliminates charging delays (Jennings and Figliozzi, 2019; Garus et al., 2024). In addition, SDRs have lower operational costs as they do not require drivers, making certain optimization strategies more feasible compared to EV-based models (Alverhed et al., 2024). By integrating SDR-specific constraints and charging strategies, our model simplifies assumptions and enhances the practicality of last-mile delivery optimization.

Moreover, energy constraints are not the only source of uncertainty in PDPTW extensions. Another critical challenge is the presence of **stochastic demand**, where the exact demand quantity is unknown at the planning stage. Unlike deterministic PDPTW models, real-world applications often require decisions to be made before demand is fully realized. This introduces significant complexity in optimizing vehicle assignments and routing strategies while ensuring service reliability. A significant contribution to this field is the development of **two-stage stochastic optimization models** for PDP with uncertain demands, which decompose decision-making into two distinct stages:

- **First stage**: Strategic decisions made before uncertainties are revealed.
- **Second stage**: Operational decisions adjusted after uncertainties become known.

Several studies have already explored PDPTW with stochastic demand. Ghilas et al. (2016) applied a Sample Average Approximation (SAA) method combined with ALNS to generate and solve demand scenarios, dynamically adjusting decisions during execution to mitigate cost and service disruptions. Mourad et al. (2021) extended the problem to PDPTW with Scheduled Lines (PDPTW-SL), leveraging a two-stage stochastic approach to optimize PD robot routes under uncertain demand. Another line of research focuses on chance-constrained optimization; for example, Wang et al. (2023) developed a branch-price-and-cut framework for PDP with Hard Time Windows (PDPHTW), incorporating node and global service level constraints to ensure robust solutions under stochastic and time-dependent travel times. Additionally, Alvarez et al. (2024) studied the consistent vehicle routing problem (ConVRP) with stochastic customer demand, incorporating penalty-based soft constraints into a two-stage stochastic model and solving it via branch-and-cut and Benders decomposition to balance consistency and operational efficiency.

Despite these contributions, existing models primarily focus on classical PDPTW settings without considering SDR-specific constraints. In this study, we extend the traditional PDPTW framework to SDR-PDPTW, addressing two key challenges:

- **Soft time windows**: Unlike hard constraints in classical PDPTW, we introduce penalty-based soft time windows, improving routing flexibility by allowing controlled violations when necessary.
- **Charging behavior integration**: We explicitly model charging stops for SDRs, balancing delivery efficiency and energy constraints.

By incorporating these elements into a two-stage stochastic model, we aim to bridge the gap between theoretical PDPTW formulations and real-world SROFD systems, providing a more robust and adaptive routing strategy under demand uncertainty.

3. Problem description and mathematical formulation

In this section, we first lay the foundation of our model by explicitly stating the underlying assumptions. Building on these assumptions, we then present the detailed mathematical formulation of the two-stage stochastic model. **The first stage** determines the optimal fleet size and the number of additional batteries, while **the second stage** focuses on solving the routing problem (SDR-PDPTW) under various demand scenarios. This section is structured as follows: Section 3.1 provides a concise summary of the problem assumptions, and Section 3.2 outlines the complete mathematical formulation including decision variables, constraints, and objective functions.

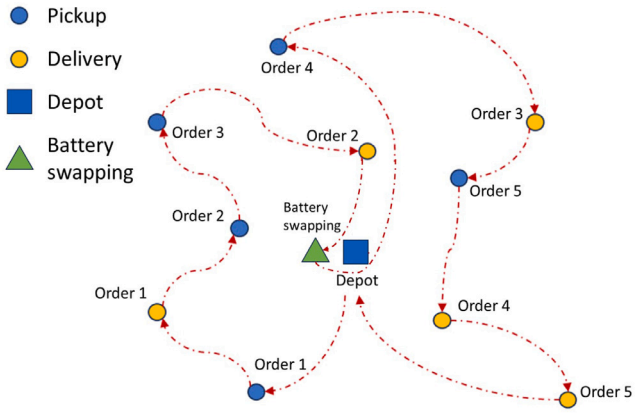

Fig. 1. Illustration of an SDR system.

Table 1
First stage notations for PDPTW.

First stage notations	Meaning
ω	Scenario
φ_ω	Probability of occurrence of ω
K	Fleet size (variable)
q	Number of additional battery (variable)
β	Unit cost of one SDR
γ	Unit cost of one battery
$\mathcal{Z}$	Total budget limit

3.1. Model assumption and problem description

To establish a clear foundation for the model, we make the following assumptions:

- **Service Area and Network Structure:** The delivery area is modeled as a graph where nodes represent the depot, pickup locations, delivery locations, and a dedicated battery swapping station. The depot serves as both the start and end point for all routes.
- **Order Characteristics:** Each customer order consists of a pickup–delivery pair. Orders are associated with time windows, which are treated as soft constraints (i.e., late deliveries incur penalties), and a pickup must always precede its corresponding delivery.
- **Homogeneous Fleet:** All SDRs have identical fixed capacity (Q) and a battery capacity (B). Battery consumption is proportional to travel time with a constant rate (μ). It is assumed that during an operational period, each SDR is allowed at most one battery swapping operation; upon swapping, the battery is restored to full capacity.
- **Stochastic Demand:** The number of orders generated in a given service period is stochastic and follows a Poisson distribution with a mean (Λ). Consequently, order demand, locations, and service time windows are subject to uncertainty.

These assumptions reflect real-world SDR constraints and distinguish our model from traditional EV-based routing problems.

Fig. 1 illustrates an example of our problem setting. The delivery area is modeled as a graph comprising $2n+3$ vertices, where vertex 0 represents the starting depot, vertex $2n+1$ represents the final destination, and vertex $2n+2$ represents the battery swapping station. The set of vertices $\mathcal{P} = \{1, \ldots, n\}$ represents pickup locations, while the set $\mathcal{D} = \{n+1, \ldots, 2n\}$ represents delivery locations. There is a fleet of vehicles $\mathcal{V} = \{1, \ldots, K\}$ available, each constrained by a maximum capacity Q and battery limit B. Each pickup location $i \in \mathcal{P}$ is associated with a corresponding delivery location $(i+n) \in \mathcal{D}$, requiring the transport of orders from pickup to delivery. Each pickup location has one order to pick up.

3.2. Mathematical formulation

We now present the mathematical formulation of the proposed two-stage problem. Tables 1 and 2 summarize all variables involved in the formulation.

Table 1 provides comprehensive definitions and justifications of the key parameters used in the first-stage optimization model. The fleet size K represents the number of SDRs required to meet the stochastic order demand within the specified operational period. Parameter q indicates the additional swappable batteries necessary to maintain continuous SDR operations, crucially influencing

Table 2
Second stage notations, parameters, and decision variables for PDPTW.

Second stage notations	Meaning
n	Number of orders in one scenario
$0,\ 2n+1$	Depot instances in one scenario
$2n+2$	Battery swapping instance in one scenario
$\mathcal{P}^\omega$	Set of vertices for pickup locations in scenario ω, $\mathcal{P}=\{1,\dots,n\}$
$\mathcal{D}^\omega$	Set of vertices for delivery locations in scenario ω, $\mathcal{D}=\{n+1,\dots,2n\}$
$\mathcal{C}^\omega$	Set of vertices for customer locations in scenario ω, $\mathcal{C}=\mathcal{P}\cup\mathcal{D}$
$\mathcal{N}^\omega$	Set of vertices for all locations in scenario ω, $\mathcal{N}=\mathcal{C}\cup\{0,2n+1,2n+2\}$
$\mathcal{N}_0^\omega$	$\mathcal{C}^\omega\cup\mathcal{D}^\omega\cup 0$
$\mathcal{N}_{2n+1}^\omega$	$\mathcal{C}^\omega\cup\mathcal{D}^\omega\cup 2n+1$
$\mathcal{V}$	set of vertices for vehicles $\mathcal{V}=\{1,\dots,K\}$
Parameters	
e_i	Earliest start time of service at node i
l_i	Latest start time of service at node i
t_{ij}	Time between node i and node j
B	Battery capacity
μ	Power consume rate w.r.t time
Q	Maximum capacity of a vehicle
ρ	Penalty per late order
M	A very big number
Decision Variables	
x_{ijk}	Link and vehicle selection, binary
τ_{ik}	Arrive time at node i for vehicle k
y_{ik}	Stop visit indicator, binary
E_{ik}	Current battery level at node i for vehicle k
ϕ_{ik}	Remaining capacity at node i for vehicle k
r_i	Late order indicator, binary

service reliability and operational efficiency. Unit costs β and γ reflect practical procurement and operational considerations. The budget limit Z sets a practical constraint ensuring cost-effective resource allocation decisions within realistic financial limitations.

The objective of the model is to optimize resource allocation decisions for establishing an SROFD system given an expected number of orders Λ generated in a period in a pre-determined area. The first-stage decision variables are the fleet size, K, and the number of additional batteries, q, both of which are integers. We define a function $\bar{C}:\mathbb{R}^3\to\mathbb{R}$ to represent the expected operational cost of the SROFD system under uncertainty, where $\mathbb{R}^3$ represents the three-dimensional input domain and $\mathbb{R}$ represents that of the expected cost. The input domain consists of the mean number of orders, Λ, fleet size, K, and additional batteries, q. The expected operational cost function $\bar{C}$ is defined as follows:

$$\bar{C}(\Lambda,K,q)=\sum_{\omega}\varphi_\omega C(\omega,K,q),\quad \omega\sim\text{Poisson}(\Lambda) \tag{1}$$

Here, $\bar{C}(\Lambda,K,q)$ represents the expected cost for a given input tuple (Λ,K,q), and $C(\omega,K,q)$ represents the operational cost under scenario ω. The probability of scenario ω occurring is denoted by φ_ω, which follows a Poisson distribution with parameter Λ. When Λ is fixed, $\bar{C}(\Lambda,K,q)$ is completely determined by the input of the resource allocation decision (K,q).

We define S as the set of possible resource allocation choices: $S=\{(K_1,q_1),\dots,(K_I,q_I)\}$, where I represents the number of allocation options. The system cost also includes the fixed procurement cost for K SDRs and q additional batteries. The cost of each SDR is β, and the cost of each additional battery is γ. A budget limit $\mathcal{Z}$ is imposed as a resource constraint.

The first-stage optimization problem can be formulated as follows.

$$\min_{(K,q)\in S}\quad(\beta K+\gamma q)+\bar{C}(\Lambda,K,q) \tag{2}$$

subject to (3)

$$\beta K+\gamma q\le\mathcal{Z} \tag{4}$$

$$K\in\mathbb{Z}^+,\ q\in\mathbb{N} \tag{5}$$

Given Λ and a limited budget $\mathcal{Z}$, the solution space S contains different combinations of (K,q) that are evaluated for their operational costs, $\bar{C}(\Lambda,K,q)$, by solving the second stage problem.

We refer to the routing problem associated with each second-stage scenario as SDR-PDPTW. We use the three-notation formulation similar to Schneider et al. (2014) to formally define the second stage problem under each scenario ω. The SDR-PDPTW is presented in Eqs. (6a)–(6u).

$$\min_{K,q}\quad C(\omega,K,q)=\sum_{i\in\mathcal{N}^\omega}\sum_{j\in\mathcal{N}^\omega}\sum_{k\in\mathcal{V}}t_{ij}x_{ijk}+\rho\sum_{i\in\mathcal{P}^\omega}r_i \tag{6a}$$

$$\text{s.t.} \quad \sum_{k\in\mathcal{V}} y_{ik} = 1 \qquad \forall i \in C^{\omega}, \tag{6b}$$

$$y_{(2n+2,k)} \le 1 \qquad \forall k \in V, \tag{6c}$$

$$\sum_{k\in\mathcal{V}} y_{(2n+2,k)} \le q, \tag{6d}$$

$$\sum_{j\in\mathcal{N}_0^{\omega}, j\neq i} x_{ijk} = y_{ik} \qquad \forall i \in \mathcal{N}_0^{\omega}, \forall k \in \mathcal{V}, \tag{6e}$$

$$y_{(2n+1,k)} = \sum_{i\in\mathcal{N}_0^{\omega}} x_{(i,2n+1,k)} \qquad \forall k \in \mathcal{V}, \tag{6f}$$

$$\sum_{i\in\mathcal{N}^{\omega}\setminus\{h\}} x_{ihk} = \sum_{j\in\mathcal{N}^{\omega}\setminus\{h\}} x_{hjk} \qquad \forall h \in C^{\omega}, \forall k \in \mathcal{V}, \tag{6g}$$

$$1 = \sum_{j\in N_{2n+1}^{\omega}} x_{0,j,k} - \sum_{j\in N_{2n+1}^{\omega}} x_{j,0,k} \qquad \forall k \in \mathcal{V}, \tag{6h}$$

$$y_{i,k} = y_{(i+n,k)} \qquad \forall i \in \mathcal{P}^{\omega}, \forall k \in \mathcal{V}, \tag{6i}$$

$$\tau_{jk} \ge \tau_{ik} + t_{ij}x_{ijk} - M\left(1 - x_{ijk}\right) \qquad \forall i,j \in \mathcal{N}^{\omega}, k \in \mathcal{V}, \tag{6j}$$

$$\tau_{(i+n,k)} \ge \tau_{ik} + t_{(i,i+n)}x_{(i,i+n,k)} \qquad \forall i \in \mathcal{P}^{\omega}, \tag{6k}$$

$$\tau_{ik} \ge e_i \qquad \forall i \in \mathcal{P}^{\omega}, \forall k \in \mathcal{V}, \tag{6l}$$

$$l_i - \tau_{ik} \le M(1 - r_i) \qquad \forall i \in D^{\omega}, \forall k \in \mathcal{V}, \tag{6m}$$

$$\tau_{ik} - l_i \le M \cdot r_i \qquad \forall i \in D^{\omega}, \forall k \in \mathcal{V}, \tag{6n}$$

$$E_{0k} = B \qquad \forall k \in \mathcal{V}, \tag{6o}$$

$$0 \le E_{ik} \le B \qquad \forall i \in \mathcal{N}^{\omega}, \forall k \in \mathcal{V}, \tag{6p}$$

$$E_{jk} = E_{ik} - \mu \cdot t_{ij} \cdot x_{ijk} \qquad \forall i,j \in \mathcal{N}^{\omega}, \forall k \in \mathcal{V}, \tag{6q}$$

$$E_{jk} = (B - \mu \cdot t_{(2n+2,j)})x_{(2n+2,j,k)} \qquad \forall j \in \mathcal{N}_{2n+1}^{\omega}, \forall k \in \mathcal{V}, \tag{6r}$$

$$\phi_{0k} = Q \qquad \forall k \in \mathcal{V}, \tag{6s}$$

$$\phi_{jk} \ge \phi_{ik} + p_i x_{ijk} - Q\left(1 - x_{ijk}\right) \qquad \forall i \in \mathcal{V}_0, \forall j \in \mathcal{V}_{2n+1}, \tag{6t}$$

$$0 \le \phi_{ik} \le Q \qquad \forall i \in \mathcal{N}^{\omega}, \forall k \in \mathcal{V} \tag{6u}$$

The objective is to minimize the total system cost given (K, q) under scenario ω. Constraints (6b)–(6h) guarantee the basic logic of SDR movements: Each customer location is visited exactly once, the battery swapping activity can occur at most once per SDR; the number of battery swapping operations needs to be less than the prepared number of additional batteries; the tour starts from the base depot; and the tour follows the conservation of flow. Constraint (6i) ensures that if a vehicle visits a pickup location, it must also visit the corresponding delivery location. Constraint (6j) ensures the time feasibility of the route and subtour elimination. Constraint (6k) is the precedence constraint for each order. Constraint (6l) restricts the pickup time being strictly later than the earliest order available time. Constraints (6m) and (6n) flag the orders that are delivered later then the expected delivery time. Constraints (6o)–(6r) define the battery restriction and depict the SOC of each SDR. Constraint (6q) guarantees that the SDR's SOC is sufficient to reach the following stop. When an SDR visits the battery swapping station, it resets its SOC to full, and its SOC at the subsequent stop corresponds to the full battery minus the energy consumption. Constraints (6s)–(6u) ensures the compartment capacity constraint of each SDR. In addition to (6b)–(6u), all variables are non-negative and the decision variables for link selection, service decision, and late order indication are binary.

The proposed stochastic program is an NP-hard problem, posing significant challenges to solve effectively. We present the details of the solution method in the following section.

4. Solution methodology

In conventional two-stage stochastic programming, solution methods typically involve iterative updates between the first and second stage (e.g., Pantelidis et al., 2024), as exemplified by the L-shaped method (Van Slyke and Wets, 1969). The sample average approximation (SAA) (Kleywegt et al., 2002) uses Monte Carlo simulation to sample different scenarios and solve for the optimal solution with sufficient sampling until convergence. Though effective, high computational effort is often required. We propose an alternative approach to solve this problem. Instead of directly solve large number of scenarios required in SAA, we sample limited number of second-stage scenarios to fit a CA model and apply that in the first stage problem. The CA model uses closed-form expressions to capture average or steady-state route performance without operation details. Such purpose suits well with the second term in the first stage model (Eq. (8)), which is the expected operation cost based on the expected route distance. In this study, the second stage model is an extension of PDPTW, whose average outcome can be well captured by CA models proposed in Figliozzi (2009) and Bergmann et al. (2020). We propose a CA model based on the formulations introduced in the two studies. Such approach

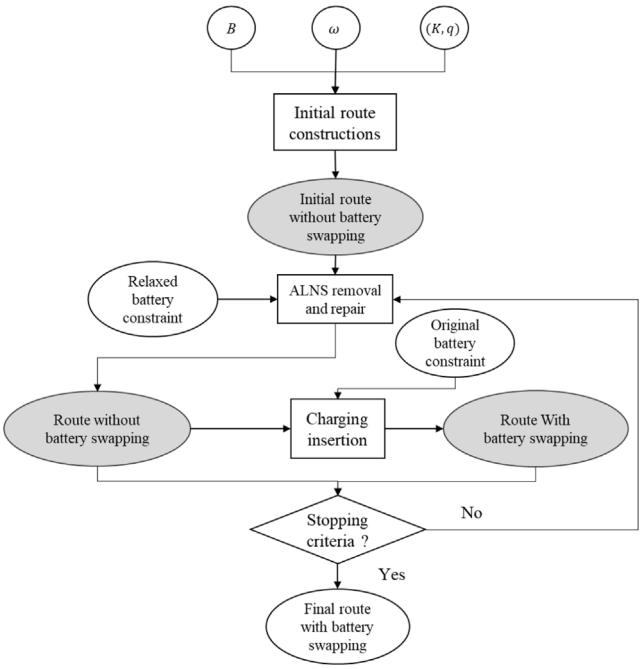

Fig. 2. Framework of the second stage solution method.

absolves the need for iterative solution method but the quality of the solution depends on how well the model fits the second stage results, and limits the problem to only scenarios that can be fitted with the CA model. The proposed solution algorithm comprises two major components: *scenario sampling* and *system cost approximation*.

The mean number of orders Λ is assumed to be known. We begin by sampling a set $\theta(\Lambda)$ of second-stage scenarios $\omega \sim \text{Poisson}(\Lambda), \omega \in \theta$, and solving each using a customized Adaptive Large Neighborhood Search (ALNS) heuristic. The sample mean of the cost function given (K, q, Λ) using a scenario set θ is defined as $\tilde{C}_\theta(\Lambda, K, q)$. A CA model $\hat{C}_\theta$ is then fitted to those sampled scenarios to approximate the function $\tilde{C}_\theta$. The fitted CA model is then used as a surrogate function to solve for the system output $(K^*, q^*) \in S$ in Eq. (2), as shown in Eq. (7).

$$(K^*, q^*) = \arg \min_{(K,q)\in S} \left[(\beta K + \gamma q) + \hat{C}_\theta \right] \tag{7}$$

The approximation solution (K^*, q^*) has three sources of error. The first source is a CA model fitting error to a data set. The second source is because a CA model only provides a point estimate of Eq. (2) based on a sample θ. The third one is from the use of a heuristic to obtain a scenario solution. The third error can be validated by comparing the heuristic performance against exact algorithm solutions using the MIP. The second error is statistical in nature and depends on sample size. The first error can be validated by sampling different sets $\theta \in \Theta$ with the (K^*, q^*, Λ) and using each set's sample average to compute Eq. (2) and obtaining a confidence interval.

Because the solution algorithm starts from solving a sample of second stage scenarios, we first present the customized ALNS based heuristic that solves the routing problem. We then present the approximation based solution method for the whole model and the additional validation step.

4.1. Scenario sampling: SDR-PDPTW routing heuristic

We develop a heuristic based on ALNS to solve the routing problem for an operation given a second stage scenario ω and a set of resources (K, q). The following subsections begin with the overall framework, followed by details of the involved heuristic algorithms. Then, we present the algorithm for constructing the initial solution without charging, which serves as the input for our ALNS. Next, we introduce the ALNS framework, including various removal and insertion algorithms. We then present the charging insertion algorithm to construct routes with charging activities. Finally, we summarize the entire heuristic process.

In the following sections, we use the terms "pickup and delivery pair" and "order" interchangeably to refer to the same concept. Fig. 2 illustrates the heuristic framework.

4.1.1. Initial solution construction

We relax the battery capacity constraint in Eq. (6p) using Eq. (8) to find an initial solution without battery swapping.

$$\hat{B} = 2(B - \bar{d}_c) \tag{8}$$

where $\hat{B}$ is the relaxed battery capacity, and $\bar{d}_c$ is the average distance between the battery swapping station and all service nodes. The relaxation is based on the assumption that only one battery swapping operation is allowed for each SDR. The order-only solution could include routes whose total distances exceed the original battery constraint, requiring battery swapping activities. Subsequently, we insert the battery swapping decisions into the routes, creating a solution bounded by the original battery constraint.

The initial solution is constructed with only order indices and the relaxed battery capacity $\hat{B}$ using a greedy insertion heuristic. It sequentially assigns tasks to vehicles while respecting capacity and time window constraints. It begins by sorting pickup nodes based on their start times to prioritize early pickups. For each vehicle, a route starting and ending at the depot is initialized. The heuristic iteratively inserts the best pickup and delivery nodes, minimizing travel time and ensuring feasibility. If no feasible insertion is found, a new vehicle is started. This process continues until all pickup nodes are assigned or no further feasible insertions can be made. The initial solution is then improved and modified in the ALNS algorithm.

4.1.2. ALNS framework

ALNS iteratively manages a series of removal or insertion actions. ALNS establishes a scoring system where each removal or insertion action is awarded a score based on the quality of the solution it produces. After a certain number of iterations, the scores are used to update the weights of the actions, which then determine the probability of selecting each action in subsequent iterations. This ensures that well-performing actions are more likely to be chosen. ALNS also employs a simulated annealing (SA) acceptance criterion to accept worse solutions with a certain probability, helping the algorithm escape local optima. The acceptance probability decreases over time according to a cooling schedule.

The removal algorithms destroy the current solution by removing a certain number of nodes, specifically the pickup and corresponding delivery nodes in our context. Different removal algorithms use various metrics to guide ALNS in removing orders. We use Shaw removal, Worst removal, and Slack induction by string removal (SISR). The first two are developed by Ropke and Pisinger (2006), while SISR, adapted from Christiaens and Vanden Berghe (2020), was originally for VRP. For SISR in the removal procedure, the objective function minimizes both the route distance and the number of late orders, so the temporal slack also needs to be considered. Instead of choosing nodes purely based on spatial closeness, the closeness between order ready/finish time is also measured.

Insertion actions reconstruct the solution by inserting orders back into the route, ensuring the pickup node is visited before the delivery node. This requirement distinguishes PDPTW from standard VRP. We use greedy insertion and regret insertion algorithms from Ropke and Pisinger (2006). After removal and repair, SA acceptance criteria update the best and accepted solutions. Rather than the current best, the accepted order-only solution serves as the base for constructing the final solution with battery swapping, expanding the solution searching space.

4.1.3. Proposed battery swapping insertion

Because the order-only solution is constructed based on the relaxed battery constraint $\hat{B}$, we allow the routes to violate the original battery constraint B. Therefore, we propose inserting the battery swapping operation to construct the solution bound by the original constraint. We first check the number of routes requiring battery swapping. If the number of routes exceeds the available number of battery q, the algorithm immediately goes to the next iteration. If it is less than q, for any route that violates the original battery constraint B, we use greedy insertion to insert the battery swapping node. After inserting at each position, two sub-routes are formed, one starts from the depot and ends at the battery swapping station, and the other starts from the battery swapping station and ends at the depot. Because the SOC resets to full after the battery swapping operation, the two sub-routes are both constrained by the original battery capacity B. Therefore, the solution is feasible when both sub-routes consume less energy than B.

Fig. 3 shows an example of the battery swapping insertion. In this example, $B = 14$ and $\bar{d} = 2.5$. According to Eq. (8), $\hat{B} = 23$. The route on the left is the solution without battery swapping obtained after the removal and repair operations, which is bounded by $\hat{B}$ while violating B. After inserting the battery swapping node, both sub-routes are bound by B. Therefore, a solution with battery swapping is obtained.

Algorithm 1 summarizes the ALNS heuristic, which also calls Algorithm 2 for the battery swapping insertion operation.

Eq. (8) constructs a route could results in an incomplete search space for final route solutions. To illustrate this, consider the extreme case shown in Fig. 4. Suppose we have a feasible route with $\hat{B} = 2B$, where the battery swapping station is located exactly at the midpoint. In this scenario, no detour is required to accommodate the battery swapping activity. However, the relaxed route violates the battery capacity constraint defined in Eq. (8), potentially excluding some feasible routes.

Although this approach may reduce the search space and risk missing the global optimum, the buffer introduced in Eq. (8) brings a practical balance between runtime efficiency and solution quality. In cases where $\hat{B} = 2B$, the additional time required for feasibility checks when all possible insertions violate the battery constraint would likely outweigh the marginal improvements in solution quality. We present experiments that showcase the efficient solution time and high solution quality in Section 5.2.

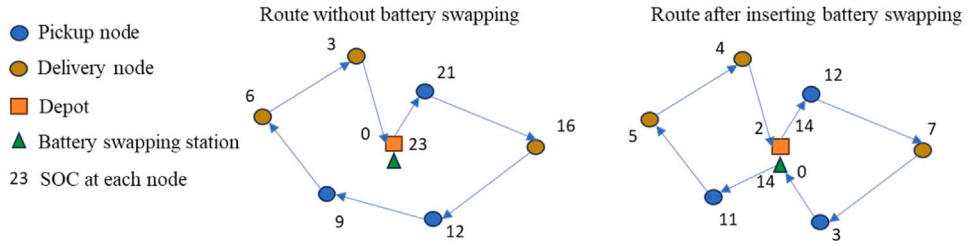

Fig. 3. Illustration of a route before (left) and after (right) battery swapping insertion.

Algorithm 1 ALNS then battery swapping insertion heuristic

Input: Initial order-only routing solution $\hat{\pi}$, maximum iterations max_iter, segment length seg_len, number of segments num_seg, learning rate ϕ, scoring parameters σ, starting temperature T_0, cooling rate ψ, number of no improvement δ
Output: Best solution with battery swapping π^*
Initialize best order-only solution $\hat{\pi}^*$ to $\hat{\pi}$
Initialize heuristic weights and scores
Initialize current temperature T to T_0
Initialize π^* to empty, the objective value v^* to ∞
for each segment from 1 to num_seg **do**
    Update heuristic weights based on scores, ϕ, and σ
    for each iteration in segment length seg_len **do**
        Select removal heuristic based on weights
        Select insertion heuristic based on weights
        Apply removal heuristic to remove requests from $\hat{\pi}$
        Apply insertion heuristic to reinsert requests into $\hat{\pi}$
        if new solution $\hat{\pi}'$ is better than $\hat{\pi}^*$ **then**
            Update $\hat{\pi}^*$ to $\hat{\pi}'$
            Assign high scores to the heuristics used
        else if New solution $\hat{\pi}'$ is better than the current solution $\hat{\pi}$ **then**
            Update current solution $\hat{\pi}$ to $\hat{\pi}'$
            Assign moderate scores to the heuristics used
        else
            Calculate acceptance probability based on current temperature
            if random number is less than acceptance probability **then**
                Update current solution $\hat{\pi}$ to $\hat{\pi}'$
                Assign low scores to the heuristics used
            end if
        end if
        Update T using cooling rate ψ
        if Less than q routes requires battery swapping in $\hat{\pi}$ **then**
            Use $\hat{\pi}$ as input, apply **Algorithm 2** to construct solution with battery swapping π
            if objective value of $\pi \leq v^*$ **then**
                Update π^* to π, v^* to objective value of π^*
            end if
            if v^* remains unchanged for δ iterations **then**
                End of the algorithm
            end if
        else
            Jump to the next iteration
        end if
    end for
end for
return best solution s^*

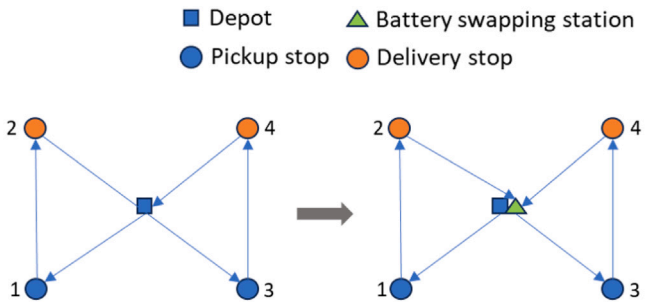

Fig. 4. Route before (left) and after (right) charging insertion without any detour.

Algorithm 2 Battery swapping insertion

Input: Current order-only solution $\hat{s}$, battery constraint B
Output: Solution with battery swapping s
Initialize $s = \hat{s}$
for route π in s **do**
  if total battery consumption $> B$ **then**
    for each position in π **do**
      Insert battery swapping node to form a new route π'
      if π' is feasible **then**
        Store the objective value of the new route π'
      end if
    end for
    Update π with the r' that has the lowest objective value
  else
    π unchanged
  end if
end for
return Solution with battery swapping s

4.2. System cost approximation

The scenario generation process follows the Poisson distribution. Assume that a convex polygon of size A contains all vertices $\mathcal{N}$. In a time frame U, the expected number of orders is Λ, and the SDR battery capacity is B. Given these values, CA models are applicable in estimating the expected total route distance. We propose a CA-based solution method inspired by Figliozzi (2009) and Bergmann et al. (2020) to construct $\hat{C}_\theta$ for a sample set θ and integrate it with Eq. (7) to obtain the final model output (K^*, q^*). Fig. 5 illustrates the whole solution method for the system cost approximation.

We denote r as the number of realized orders and $v_r^{(K,q)}$ as the mean operating cost for all possible scenarios with r orders given (K, q). Given Λ, the upper and lower bounds of r can be determined as $[\underline{r}^\Lambda, \bar{r}^\Lambda]$, where $\underline{r}^\Lambda$ and $\bar{r}^\Lambda$ correspond to the lower and upper 0.25th percentiles such that the considered order volume covers 99.5% of the distribution. The probability of a scenario having $r \in [\underline{r}^\Lambda, \bar{r}^\Lambda]$ orders is φ_r^Λ. Therefore, $\bar{C}(\Lambda, K, q)$ can be calculated using Eq. (9).

$$\bar{C}(\Lambda, K, q) = \sum_{r \in [\underline{r}^\Lambda, \bar{r}^\Lambda]} \varphi_r^\Lambda v_r^{(K,q)} \tag{9}$$

For the sake of simplicity, we use $\underline{r}$ and $\bar{r}$ to represent $\underline{r}^\Lambda$ and $\bar{r}^\Lambda$ given Λ. The solution method involves two phases: the fitting phase and the approximation phase. The fitting phase uses a sampling set $\theta(\Lambda)$ to find the full solution space S and the CA model $\hat{C}_\theta$ for accurately approximating $v_r^{(K,q)}$. The approximation phase applies the $\hat{C}_\theta$ to obtain the expected cost given $r \in [\underline{r}, \bar{r}]$ for all $(K, q) \in S$.

We expand the CA model $\hat{C}_\theta$ as function $\hat{C}_\theta(\Lambda, A, U, B) : (K, q, r) \to v_r^{(K,q)}$. With the same Λ, A, U, and B, same fitted $\hat{C}_\theta(\Lambda, A, U, B)$ can be directly applied to approximate $v_r^{(K,q)}$ for different combinations of (K, q) and demand level r. Changes in the values in Λ, A, U, or B would lead to refit $\hat{C}_\theta(\Lambda, A, U, B)$. We presents the CA fitting process in the following contexts.

For the whole solution space S, the upper bound of K is determined by both the budget limit $\mathcal{Z}$ and a hypothetical boundary $\hat{K}$. If $\mathcal{Z} = \infty$, the sole upper bound of K is $\hat{K}$. When $K > \hat{K}$, the expected total cost calculated from Eq. (2) is dominated by the solutions with $K \le \hat{K}$. Therefore, we need to approximate the value of $\hat{K}$ to initialize the process.

We first estimate the upper bound of the fleet size required for each scenario ω. We propose that when facing scenario ω with the total number of orders $|\mathcal{P}^\omega|$, the upper bound of the fleet size $\hat{K}_\omega$ is the fleet size required under the PDPTW constraints. We formally present the concept in the following proposition.

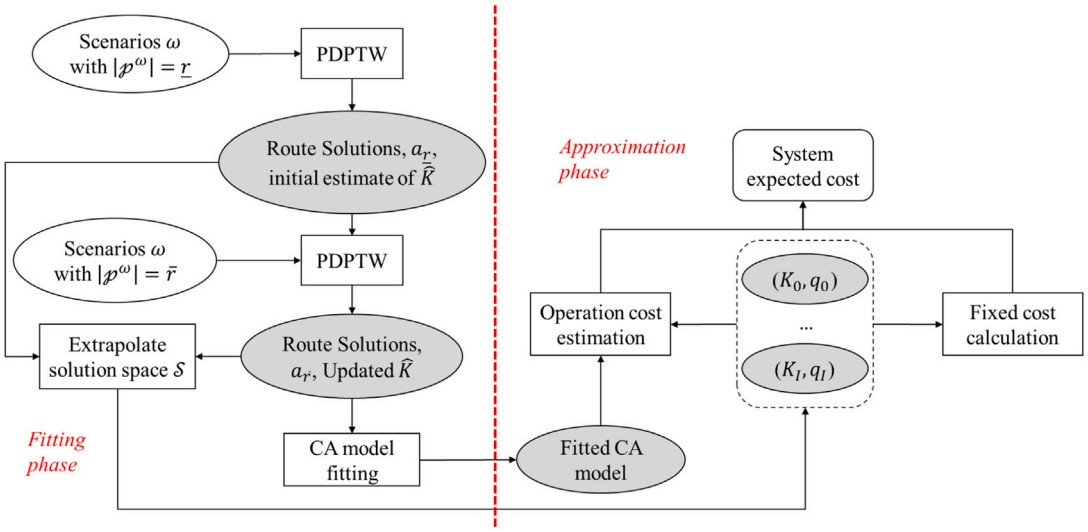

Fig. 5. Framework of the model solution method for system cost approximation.

Proposition. *Given a scenario ω of a SDR-PDPTW, the upper bound of the required fleet size $\hat{K}_\omega$ is the optimal number of SDRs required to solve the corresponding PDPTW problem.*

Proof. The PDPTW problem requires all orders to be served within hard time-window constraints. Suppose $\hat{K}_\omega$ SDRs are needed to meet the requirements of PDPTW and all SDRs can complete their routes without battery swapping.

Allowing some flexibility in the hard time window enables SDRs to find more efficient routes, potentially increasing the average number of deliveries per SDR despite battery constraints. Therefore, a new fleet size $\bar{K}_\omega \le \hat{K}_\omega$. Similarly, if each SDR can receive a battery swapping operation, orders served by two SDR routes can potentially be integrated into one. Therefore, the new fleet size $\bar{K}_\omega \le \hat{K}_\omega$. This concludes the proof. □

As $r \in [\underline{r}, \bar{r}]$, when $|\mathcal{P}^\omega| = \bar{r}$, the required fleet size $\hat{K}_\omega$ is largest in all simulated scenarios. We denote this simulated scenarios as $\hat{\omega}$. Therefore, $\hat{K} = \hat{K}_{\hat{\omega}}$. Now we present the process of estimating $\hat{K}_r$ and the maximum fleet size $\hat{K}$.

We first simulate a set of scenarios $\omega \in \Omega_1$ with $\underline{r}$ orders and solve them under the PDPTW constraints. The average number of orders $a_{\underline{r}}$ that an SDR can handle is obtained by averaging the sampled routing results. We apply Eq. (10) to obtain an initial estimate of $\hat{K}_{\underline{r}}$, which is the average number of orders each SDR handles for all scenarios ω with $|\mathcal{P}^\omega| = \underline{r}$.

$$\hat{K}_{\underline{r}} = \left\lceil \underline{r}/a_{\underline{r}} \right\rceil \tag{10}$$

When r increases from $\underline{r}$ to $\bar{r}$, the density of the order increases both spatially and temporally, resulting in a higher level of a_r. For a more accurate estimate of a_r under higher demand level, we simulate a new set of PDPTW scenarios $\omega \in \Omega_2$ with demand level $\bar{r}$ and obtain the routing solutions and $a_{\bar{r}}$. By applying linear interpolation with $a_{\underline{r}}$ and $a_{\bar{r}}$, we can estimate a_r at each demand level r. We update all $\hat{K}_r$ using Eq. (10) and set $\hat{K} = \hat{K}_{\bar{r}}$. The initial solution search space for K is therefore $[0, 1, \dots \hat{K}]$.

In addition to the previously obtained PDPTW solutions from the two sets of scenarios Ω_1 and Ω_2, we further generate and solve PDPTW scenarios with $r \in [\underline{r}, \bar{r}]$ to fit the proposed CA model. The total number of scenarios involved for fitting purpose ranges from 50 to 100 based on the scale of the scenario. This balances the trade-off of the fitting quality and the computational requirement. With U and B fixed, the total route distance L can be estimated as a closed-form formulation using demand level r and fleet size K as input variables as proposed in Figliozzi (2009) and derived in Yang et al. (2025). Based on the formulation introduced in the two studies, a CA model estimating the average tour length of PDPTW can be written as Eq. (11).

$$L = 1.62262((r - K)/r)(\alpha_1 \sqrt{A \times r}) + \alpha_2 2K\bar{d}_0 \tag{11}$$

where α_1 and α_2 are the fitted coefficients and $\bar{d}_0$ is the average distance between the stops and depot. For details of the model derivation, readers can refer to Yang et al. (2025). The fitted Eq. (11) is the CA formulation used to estimate the total PDPTW route distance given r and K under the condition of U and B.

The second stage model is an specific E-PDPTW. Therefore, we propose a CA model dedicated to the second stage model by extending the fitted Eq. (11). At demand level r with K SDRs, the total route distance of the second stage model L_r^k can be approximated using Eq. (12), which serves as the core of the CA model used in the approximation phase.

$$L_r^K = 1.62262([r - \min(K, \hat{K}_r)]/r)(\alpha_1 \sqrt{A \times r}) + 2 \times (\alpha_2 \min(K, \hat{K}_r)\bar{d}_0 + \max(\hat{K}_r - K, 0)\bar{d}_0) \tag{12}$$

The first and second terms capture the total route distance. When $K > \hat{K}_r$, fleet size $\hat{K}_r$ SDRs are sufficient to optimally serve all orders under all possible scenarios without charging, and Eq. (11) is identical to Eq. (12). When $K \leq \hat{K}_r$, only K SDRs can be deployed for service. The first and second terms represent the total route distance without battery swapping. The third term captures the additional distance induced by battery swapping operations. When the fleet size is $\hat{K}_r$, none of the SDRs requires battery swapping. By removing one SDR, one battery swapping operation is added to the remaining fleet, requiring one additional return trip to the depot. Therefore, the total additional distance is estimated by the third term.

The estimated L_r^K represents the first term of Eq. (6a) at demand level r with given fleet size K. When $\hat{K}_r \leq K$, all orders are served within the expected delivery time, inducing zero penalty cost. When $K \leq \hat{K}_r$, delivery time could be violated, causing an additional penalty cost. We use Eq. (13) to approximate the late-order penalty cost.

$$\mathcal{L}_r^K = \rho((r/\hat{K}_r)(\hat{K}_r - K)) \times ((\hat{K}_r - K)/K) \tag{13}$$

The term $r/\hat{K}_r$ captures the number of orders each SDR serves under the PDPTW constraints. By removing $(\hat{K}_r - K)$ SDRs from the fleet, the number of orders requiring to be integrated in the remaining routes is therefore $(r/\hat{K}_r)(\hat{K}_r - K)$. We assume that the occurrence of each late order follows a binary distribution. The probability of a late order is approximated by using the proportion between the removed and remaining fleets $(\hat{K}_r - K)/K$. The expected number of late orders is calculated by Eq. (13). Because the probability cannot exceed 1, the lower bound of K at each demand level r is therefore $\hat{K}_r/2$. To properly serve all scenarios even with demand level $r = \hat{r}$, the lower bound of the whole solution space is therefore $\lceil \hat{K}/2 \rceil$. We assume a unit cost per route distance η is used to capture the distance based cost. Therefore, the CA model for operating cost can be represented using Eq. (14).

$$\upsilon_r^{(K,q)} \approx \hat{C}_\theta(K, q, r) = \eta L_r^K + \mathcal{L}_r^K \tag{14}$$

Eq. (14) depends only on the input of K. The routing results directly reflects the battery capacity constraint during the fitting phase. When $K \leq \hat{K}_r$ under demand level r, additional detour caused by charging operations is captured by the third term in Eq. (12). Therefore, the additional q batteries for swapping is $(\hat{K} - K)$. This is based on the assumption that the system requires no service failure for all scenarios when $\underline{r} \leq |P^\omega| \leq \bar{r}$. When $q < (\hat{K} - K)$, SDRs could fail to complete their routes before returning to the depot. In such case, additional recourse actions are required, which can be further incorporated in the model in future. In this study, we simply eliminate all choices with $q < (\hat{K} - K)$. The value of q directly change the fixed cost term in Eq. (7). To minimize system cost, $q = (\hat{K} - K)$, and this provides us the solution set (K, q). The final solution (K^*, q^*) that produces the lowest level of expected system cost is selected as the final output. Algorithm 3 summarizes the solution method.

Algorithm 3 CA-based heuristic for the whole model

Input: Area size A, battery constraint B, expected number of orders Λ, order generation time frame U, number of sample scenarios x_1, x_2
Output: Solution S^*
Enumerate demand range $[\underline{r}, \bar{r}]$ and probability φ_r^Λ
Simulate a set of scenarios Ω_1 containing x_1 scenarios with order demand $\underline{r}$ in A and T, formulate the scenarios as PDPTW
Solve all scenarios of PDPTW in Ω_1 using Algorithm 2, obtain the average order per SDR $a_{\underline{r}}$
Obtain initial estimation of the upper bound of fleet size $\hat{K}$ using Eq. (10), $\bar{r}$, and $a_{\underline{r}}$
Simulate a set of scenarios Ω_2 containing x_2 scenarios with order demand $r = \bar{r}$, formulate them as PDPTW instances
Solve x_2 number of PDPTW using Algorithm 2, obtain $a_{\bar{r}}$
Use the PDPTW solutions to fit α_1 and α_2 using Eq.(11)
Calculate $\hat{K}$ using $\bar{r}$, $a_{\bar{r}}$, and Eq. (10)
for Demand level r in $[\underline{r}, \bar{r}]$ **do**
    Interpolate a_r based on $a_{\underline{r}}$, $a_{\bar{r}}$, $\bar{r}$, and $\underline{r}$,
    Approximate $\hat{K}_r$ at demand level r using Eq. (10)
    for Fleet size $\lceil \hat{K}/2 \rceil \leq K \leq \hat{K}$ **do**
        Use Eq. (12) to approximate expected route distance L_r^K
        Use Eq. (13) to approximate delay order penalty $\mathcal{L}_r^K$
        Calculate q corresponding to K
        Use Eq. (14) to approximate expected operating cost $\upsilon_r^{K,q}$
    end for
end for
for Fleet size $\lceil \hat{K}/2 \rceil \leq K \leq \hat{K}$ **do**
    Apply Eqs. (9) and (2) to calculate expected system cost with K and $q = \hat{K} - K$
end for
Choose the solution (K^*, q^*) with lowest expected system cost
return Final solution (K^*, q^*)

5. Experiments

5.1. Instances generation

To thoroughly evaluate our proposed model and solution method, we developed a custom instance generator tailored to the unique characteristics of SDR operations. This generator creates realistic scenarios that accurately reflect DSRs' operational constraints and environment, which differ significantly from traditional vehicle routing problems.

The instance generator requires the following inputs:

- Number of orders (pickup-delivery pairs)
- Size of the map (defining the geographical area)
- Constant speed of the robots
- Tolerance for delivery times (additional time allowance)
- Penalties for delayed or unserved orders
- Random seed (for reproducibility)

For each instance, the generator outputs:

- Coordinates of the depot, pickup points, and delivery points
- Distance matrix
- Time matrix (calculated as distance divided by speed)
- Random demands for each pickup node
- Random time windows for each node
- Random service times for each node

The demand generation process is simulated as follows:

1. Randomly select pairs of pickup and delivery locations based on a predefined probability distribution that reflects the likelihood of orders between different areas.
2. For each selected pair, generate a demand quantity using a truncated Poisson distribution, considering the robot's capacity constraints.
3. Assign a random order time within the specified time frame to each generated demand.

For an instance with n orders, we generate a list of $(2n + 3)$ indices: indices 0 and $(2n + 1)$ represent the depot (start and end points), index $(2n + 2)$ represents the charging station, indices 1 to n represent pickup nodes, and indices $(n + 1)$ to $2n$ represent delivery nodes.

To mimic real-world scenarios, we set the following parameters based on practical considerations (Wevolver, 2024):

- Coordinates generation: Uniform random distribution within the defined area
- Order generation time frame: 2 hour period (simulating lunchtime delivery peak)
- Expected delivery time: 60 minutes after order placement
- Service times: Randomly generated between 5 and 10 minutes
- Robot capacity: 6 orders
- Robot battery capacity: Sufficient for a 10-mile journey
- Robot speed: 4 miles per hour

This instance generator allows us to create diverse, realistic scenarios that capture the essence of SDR operations, enabling a comprehensive evaluation of our proposed model and solution method across various operational conditions. By adjusting the input parameters, we can simulate different urban environments and demand patterns, providing a robust testing framework for our algorithms.

5.2. Routing heuristic for second stage scenarios

Different ALNS-based heuristics have been comprehensively studied and validated in past studies, showing their efficacy in solving small to medium-sized VRPs. However, the applications of solving the PDPTW with battery swapping problem have rarely been explored. We conduct experiments to validate the solution quality of the proposed heuristic by comparing its outputs with the solutions obtained from solving Eqs. (6a)–(6u). For the removal and repair operations in our heuristic algorithm, we use parameters recommended from Ropke and Pisinger (2006) and Christiaens and Vanden Berghe (2020). The heuristic and the discrete model are run under the Windows 11 environment using Intel i9-12900h. The discrete model is compiled and solved using Gurobi 11 and the run time is restricted to 7200 s.

Due to the complexity of the problem, the discrete model fails to produce any feasible solutions for scenarios larger than 13 orders. It fails to reach the optimality condition before reaching the time limit when the scenario includes more than 10 orders. We

Table 3
Order levels and average solution gaps for each order level.

# Orders	Avg. Obj. Heuristic.	Avg. Obj. Gurobi	Avg. Time. Heuristic (s)	Avg. Time. Gurobi (s)	Gap
10	33.74	35.02	1.32	7200	3.65%
11	34.67	36.08	3.75	7200	3.93%
12	35.26	37.87	10.84	7200	6.88%
13	37.74	40.91	25.17	7200	7.75%

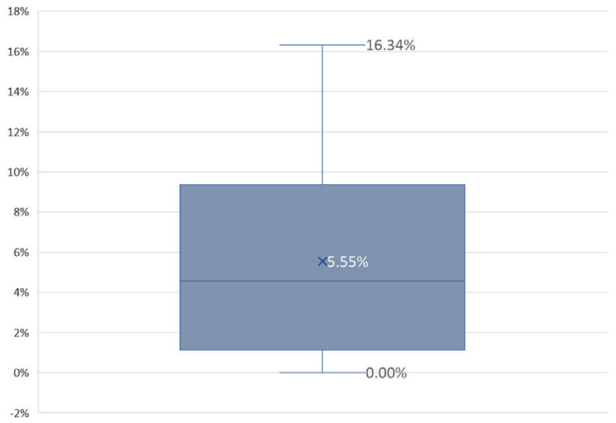

Fig. 6. Boxplot of the relative result gaps of all scenarios (the discrete model result being the baseline).

generated instances with order levels from 10 to 13. At each order level, we generate 10 random scenarios within a 4-mile by 4-mile squared area with different order locations. Therefore, we have 40 simulated scenarios in total. The orders are randomly generated within a 2 hour period, and each order is expected to be delivered no later than 60 minutes after the order generation time.

Table 3 illustrates the average objective values obtained from using the proposed heuristics and Gurobi along with corresponding runtime. Fig. 6 illustrates the distribution of the relative result gaps shown in all instances. Out of the 40 instances among the order levels (10 to 13), results obtained from using heuristics are either equal to or better than the discrete model's results obtained within the 2 hour run time limit. The solution improvement can be as large as 16.34%. All results are obtained in under 1 minute when using the proposed heuristic. Though the test instances are limited, they still demonstrate that the proposed heuristic can effectively provide high-quality solutions for small to medium-sized instances. The solution algorithm can be further improved with more efficient coding logic, but this is outside of this study's scope. More comprehensive validation processes will be conducted in future work.

5.3. Two-stage stochastic model

We conduct experiments regarding the whole solution method. Although the CA model provides a good point estimate of the expected value of the system outcome, it lacks the ability to account for the variability inherent in the stochastic model. In addition, the fitting error caused by the limited sample could also lead to inaccurate estimate. Therefore, we include validation steps in our experiments. In addition, we apply the traditional SAA approach to solve the same problem as a benchmark for comparison.

For validation, we include a solution space $\tilde{S}^*$ so that: $(K^*, q^*) \in \tilde{S}^* \subset S$. All $(K, q) \in \tilde{S}^*$ produce similar levels of approximated system cost using the proposed CA model. To further evaluate the system outcome for a given (K, q), we simulate additional y_1 iterations of scenarios and each iteration contains y_2 scenarios. The average system costs obtained from the additional $y_1 \times y_2$ scenarios further validates the CA model result. We denote the system cost obtained from the CA model as $\hat{Y}_{\Lambda}^{(K,q)}$ and the average system cost generated from the validation process as $\tilde{Y}_{\Lambda}^{(K,q)}$. We use the variance of the mean across all scenarios to evaluate the variation embedded in the cost of the stochastic system. Let $\varepsilon^{(K,q)}$ represent the variance across the $y_1 \times y_2$ obtained system costs; the variance of $\tilde{Y}_{\Lambda}^{(K,q)}$ can be estimated as $\varepsilon_Y^{(K,q)} = \varepsilon^{(K,q)}/y_2$.

After obtaining $\tilde{Y}_{\Lambda}^{(K,q)}$ and $\varepsilon^{(K,q)}/y_2$, the statistical properties of the solutions can be analyzed to gain a better understanding of the impact of (K, q) under uncertainty. In the following sections, we conduct extensive numerical experiments and case studies to demonstrate both the solution algorithm and the validation results.

For the experiment, we randomly generate 10 pickup locations (e.g., restaurants) and 10 delivery locations (e.g., apartment complexes) to form 100 pickup-delivery pairs in a 2.5 mile by 2.5 mile squared area. Orders are generated during a 2 hour period. The battery swapping facility locates inside the depot. Fig. 7 shows the layout of the area. We assume the daily unit cost per SDR is \$7, the operation cost per mile is \$0.1, the daily additional battery cost is \$2, and the penalty per late order is \$2. We assume that there is no budget limit $\mathcal{Z}$, resulting in very large solution search space S. We demonstrate the effectiveness of the solution method by applying this extreme case.

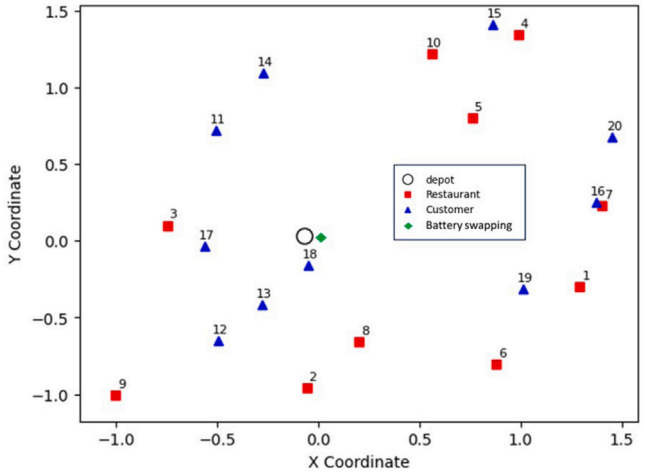

Fig. 7. Node locations included in the experiment.

Table 4
Estimated total expected costs for various fleet sizes and additional battery quantities.

Fleet size (K)	Additional batteries (q)	$\hat{Y}_{\Lambda}^{(K,q)}$	$\tilde{Y}_{\Lambda}^{(K,q)}$
5	4	52.11	51.15
6	3	53.65	53.23
7	2	57.77	56.39
8	1	62.66	Pre-eliminate
9	0	67.66	Pre-eliminate

We first generate 50 scenarios with $\Lambda = 15$ and solve the corresponding PDPTW. The initial estimated $a_{\underline{r}} = 4.85$. For the experiment, we explore the scenarios when $\Lambda = 30$. We obtain the probability distribution from demand level 15 to 45 based on Poisson distribution, which corresponds to the 25th percentile on both ends. We then obtain an initial estimated of $\hat{K}$ using Eq. (10) with $\hat{r} = 45$ and $a_{\underline{r}} = 4.85$, which results in $\hat{K} = 10$. We simulate 10 additional scenarios with $r = 45$ and solve the PDPTW given $K = 10$. The value of $a_{\bar{r}}$ is 5.5. Based on $a_{\underline{r}}$ and $a_{\bar{r}}$, we interpolate a_r for each demand level $r \in \{15, 16, \ldots, 45\}$ and calculate corresponding $\hat{K}_r$ using Eq. (10). As a result, $\hat{K} = \hat{K}_{45} = 9$. We simulate another 40 scenarios with $r \sim Poisson(30)$ and solve them exactly for fitting purpose. The 100 scenarios and solutions are used to fit Eq. (12), from which we obtain the fitted value of $\alpha_1 = 1.2$ and $\alpha_2 = 2.1$ under the given conditions.

The complete solution space of the first stage is shown in Table 4. When $K = 5$ and $q = 4$, the estimated total expected cost is the lowest. Therefore, ($K = 5$, $q = 4$) can be selected as the final output. Because the associated system cost given ($K = 6$, $q = 3$) is also close to ($K = 5$, $q = 4$), we conduct an additional validation process to better evaluate the robustness of the result.

We involve 4 iterations, and each iteration includes 25 scenarios. A total of 100 scenarios are simulated for each of the selected (K, q). In addition to ($K = 5$, $q = 4$) and ($K = 6$, $q = 3$), we also simulate another 100 scenarios for $(K = 7, q = 2)$ to further validate the first stage result. The average total cost with $(K = 6, q = 3)$ is \$53.23 across the 100 scenarios. When $(K = 5, q = 4)$, the average cost is \$51.15. The average cost is \$56.39 when $(K = 7, q = 2)$. The average total costs obtained from the validation scenarios are very close to the approximated CA result, and the average system cost when $(K = 7, q = 2)$ is dominated by the other two solutions. Because of the close proximity between $\hat{Y}_{\Lambda}^{(K,q)}$ and $\tilde{Y}_{\Lambda}^{(K,q)}$, we do not list out the variance. The best resource allocation strategy is to acquire 5 SDRs and 4 additional batteries when serving the area with $\Lambda = 30$ in a two-hour period.

We use the classic SAA method as a benchmark. However, because our two-stage model relies on the first-stage decision variable to generate the second-stage solution, we adopt a modified SAA approach that deviates slightly from the method described in Kleywegt et al. (2002). First, we simulate 50 random scenarios with $r = 45$ and solving them to generate an initial set of candidate solutions using a grid of inputs as starting points. Next, using a fixed random seed, we simulate scenario samples with $r \sim Poisson(30)$ for each candidate resource allocation decision. After every batch of 100 scenarios, we update the candidate set by incorporating new inputs and eliminating dominated solutions. This iterative process continues until one decision consistently produces the lowest average first-stage objective value with a clear margin over the others.

Table 5 summarizes the candidate solutions obtained through the benchmark SAA process. The candidate solutions generated from the initial 100 simulated scenarios are identical to those produced by the CA-based approach. In addition, the table presents the average system cost computed over 100 and 200 simulated scenarios, denoted as $\bar{Y}100$ and $\bar{Y}200$, respectively. Due to the modest scale of the test case, the solution converges quickly as more scenarios are incorporated. The table also lists the percentage differences between the CA-based estimates (denoted as $\tilde{Y}$ for simplification) and the SAA results (based on 200 sampled scenarios), demonstrating that the CA-based estimates closely match the SAA results across all candidates.

Table 5
Average expected cost for fleet size and additional battery quantity based on classic SAA solutions.

Fleet size (K)	Additional batteries (q)	$\tilde{Y}_{100}$	$\tilde{Y}_{200}$	% difference between $\tilde{Y}_{200}$ and $\tilde{Y}$
5	4	51.39	51.29	1.57%
6	3	54.27	54.18	−0.99%
7	2	58.67	58.61	−1.45%
8	1	63.62	63.49	−1.32%
9	0	67.63	67.58	0.12%

Table 6
Estimated total expected costs for various fleet sizes and additional battery quantities.

Fleet size (k)	Additional batteries (q)	$\hat{Y}_{\Lambda}^{(K,q)}$	$\tilde{Y}_{\Lambda}^{(K,q)}$, $\varepsilon_{\tilde{Y}}^{(K,q)}$
7	6	79.59	77.06, 8.70
8	5	77.70	77.32, 3.29
9	4	79.42	78.71, 3,23
10	3	83.16	Pre-eliminate
11	2	87.80	Pre-eliminate
12	1	92.72	Pre-eliminate
13	0	97.71	Pre-eliminate

Table 7
Coordinates of the corner points.

	West	East
North	(40.427371, −86.930548)	(40.427371, −86.905710)
South	(40.420486, −86.930548)	(40.420486, −86.905710)

Notably, the proposed CA approach relies on only 100 simulated samples (with half being at a smaller scale), resulting in a significantly shorter runtime. In contrast, the classic SAA approach requires solving the simulated scenarios at least 600 times in total, comprising 100 initial scenarios plus an additional 100 scenarios for each candidate solution. Consequently, even with parallel computing, the SAA algorithm demands a substantially longer runtime.

We further explore the scenarios when $\Lambda = 50$ in the same area. The new demand levels range from 30 to 70, which have an accumulated probability of 0.996. We use the value of a_{45} to be 5.5 at demand level $r = 45$ based on previous results. The new estimate of $\hat{K}$ equals to 13 when $r = 70$. We simulate 10 scenarios with $r = 70$ and solve the PDPTW. The updated $a_{\bar{r}}$ equals to 5.8. We interpolate the value of a_r and update $\hat{K}_r$ for demand levels $r \in \{45, 46, \ldots, 70\}$ while keeping the previously calculated $\hat{K}_r$ for $r \in \{30, 31, \ldots, 44\}$. The full solution space and their corresponding expected total cost is summarized in Table 6. As a result, $(K = 8, q = 5)$ is the model output. Because the solutions with $K = 7, 8, 9$ produce fairly close expected system cost, we conduct additional validation process using the previously stated structure. The average total costs are \$77.06, \$77.32, and \$78.71. Though the cost is marginally higher for $(K = 8, q = 5)$ using the added scenarios, lower variance associated with the estimated mean cost indicating a higher certainty toward the first stage approximation. Combining the two results, we can safely state that the optimal solution would be $(K = 8, q = 5)$.

6. Case study

After validating the heuristic algorithm and the two-stage model's solution approach, we conducted a case study using real-world data from the Purdue University campus in West Lafayette, Indiana. Starship has already deployed delivery robots on the Purdue University West Lafayette campus, with robots starting from the Purdue Memorial Union (PMU), completing deliveries, and then returning to PMU. However, the service area is limited to a small portion of the campus. In this section, we utilize the 'OSMnx'(Boeing, 2017) package to select relevant points of interest within a defined area centered around PMU (Wiki, 2024), which is also served as the depot for SDRs. We use the coordinates shown in Table 7 to extract the geospatial information.

Within this area, we identified school buildings, restaurants, and apartments. We used the coordinates of these locations and calculated the shortest path between each pair of points using the 'NetworkX' (Hagberg et al., 2024) library to obtain the distance matrix. This matrix serves as the input for our algorithm to perform testing. In total, 27 restaurants and 135 delivery points are selected in the 1.6 square miles area. Fig. 8 illustrates the selected area with all the selected service points. Both the apartment and university buildings are used as potential delivery stops.

We use the same metrics as in Section 5.3 for robot configuration, with a further reduction in battery capacity to 6 miles. The model is solved under two average demand levels: one with $\Lambda = 50$ and the other with $\Lambda = 70$, both within a 2 hour order generation period. Fig. 9 shows the expected total cost, approximated using the stage 1 algorithm. We also conduct a comprehensive validation step with 100 simulated scenarios for each (K, q) combination under the two Λ values and plot the average system cost along with the stage 1 expected cost. The trends between stage 1 and stage 2 results align well, demonstrating the reliability of stage 1 results.

Concave curves for expected total cost emerge as fleet size increases and the number of allowable battery swaps decreases. This aligns with the general logic of food delivery services: a larger fleet incurs higher fixed costs but minimizes delays, while a smaller

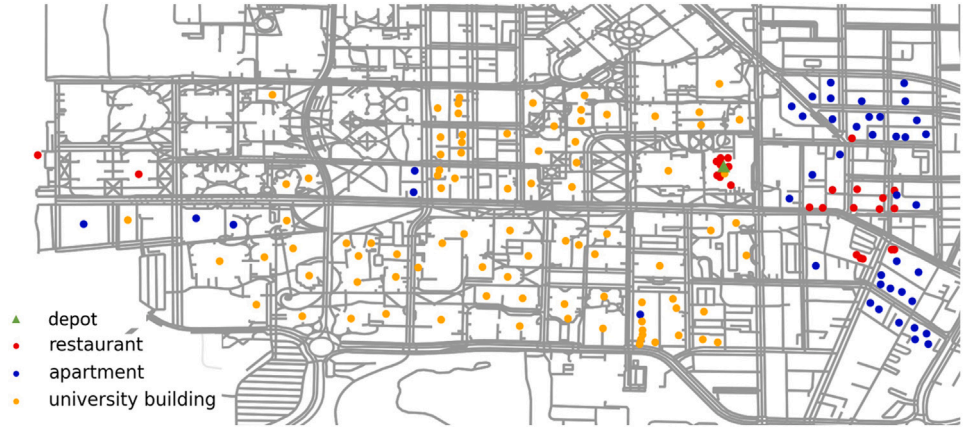

Fig. 8. Case using Purdue University Campus.

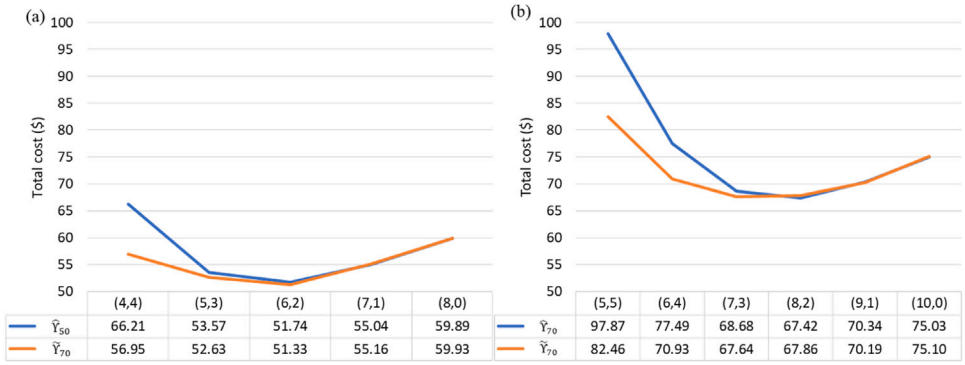

Fig. 9. Final results of instances when (a) $\Lambda = 50$ and (b) $\Lambda = 70$.

fleet saves on fixed costs but incurs higher delay penalties. In this case study, the optimal configuration for an average demand of 50 orders within 2 hours is a fleet of 6 SDRs with 2 additional batteries for swapping, resulting in an expected cost of $51.33. For an average demand of 70, the expected total costs for fleets of 7 and 8 SDRs in the stage 2 solution are similar. Because the validation step shows a lower average system cost when $(K = 8, q = 2)$, we choose $(K = 8, q = 2)$ to be the final output instead. The case study demonstrates the effectiveness of the proposed approximation method for the model. However, when the estimated system costs are too similar, further validation is still required.

7. Sensitivity analysis

We conducted a sensitivity analysis using the Purdue campus case to examine how changes in average demand level Λ and battery capacity affect fleet size and additional battery decisions. We varied Λ from 40 to 80 in increments of 10 and tested two types of SDRs: one with a 8 mile battery range and another with an 10-mile range. For stage 1 approximation, we simulate 20 scenarios for fitting purposes. After obtaining the estimated system cost using the approximation method, we select at most two (K, q) sets for validation if needed. For each combination of Λ and battery capacity, we ran 5 additional iterations of 30 simulated scenarios each for validation.

Table 8 presents the validation results after solving 150 scenarios for each combination of inputs. In addition to point estimates, we calculated the variance of the sample mean. Fig. 7 displays the mean total cost for each Λ and (K, q), with shaded areas representing the 95% confidence interval based on the sample mean–variance.

Fig. 10 reveals some notable patterns. In Fig. 10(a), the shaded area is considerably wider at lower demand levels with smaller fleet sizes. At lower demand levels, orders are more spread out spatially and temporally, reducing routing efficiency for each SDR. Small fleet size increase the likelihood of service delays, leading to a higher risk of customer dissatisfaction despite similar expected costs across fleet sizes. As Λ increases, routing efficiency improves, lowering cost variability for smaller fleets. This variance effect is less pronounced with larger battery packs, as each SDR can handle more orders per route, reducing the required resources and increasing the variance in order delays.

The final (K, q) decisions are shown in Fig. 11. Interestingly, a larger battery pack does not significantly alter the optimal fleet size but requires additional batteries to facilitate more frequent swapping, given the shorter range of the smaller battery.

Table 8
Fleet size, additional battery, mean, and variance of sample mean for 8-mile and 10-mile Battery Capacities under different order demand levels; the final results are highlighted in bold.

	Λ	(K, q)	$\hat{Y}_{\Lambda}^{(K,q)}$	$\tilde{Y}_{\Lambda}^{(K,q)}$	$\epsilon_{Y}^{(K,q)}$
	40	(4, 2)	37.01	36.28	0.66
Battery	50	(5, 2)	44.11	43.58	0.24
capacity:	60	(6, 2)	51.66	50.94	3.64
8-mile	70	(6, 3)	56.57	56.22	4.34
	80	(7, 3)	65.90	65.22	8.86
	40	(4, 1)	34.94	34.53	3.48
Battery	50	(5, 1)	41.81	43.63	12.31
capacity:	60	(5, 1)	44.47	46.33	8.23
10-mile	70	(6, 1)	50.83	51.40	1.91
	80	(7, 1)	57.92	61.62	13.98

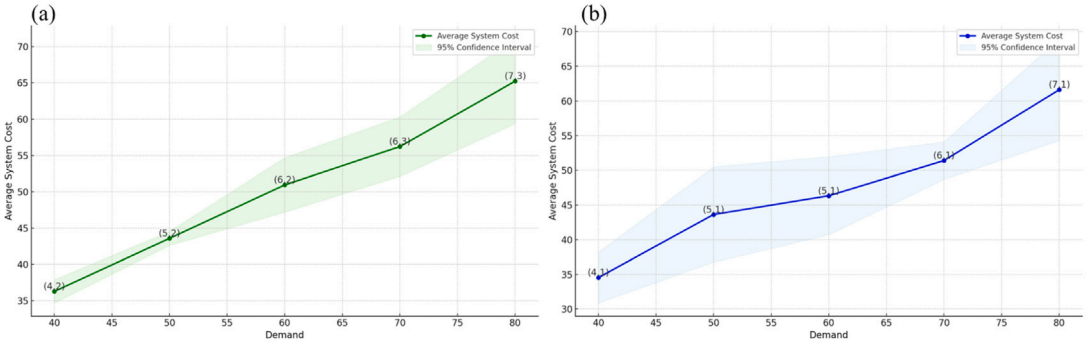

Fig. 10. Mean and 95% confidence interval of system cost from added samples of scenarios with (a) 8-mile and (b) 10-mile battery capacity.

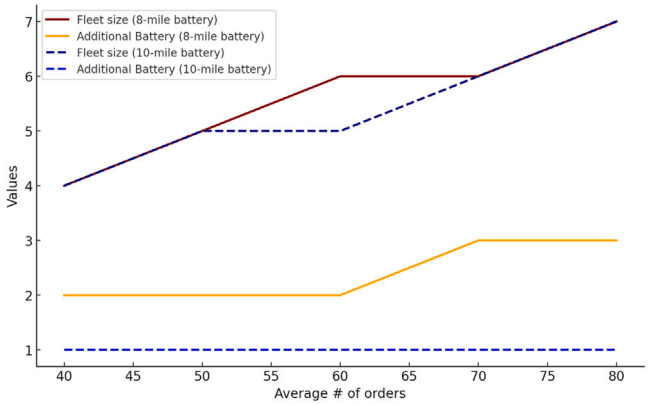

Fig. 11. Optimal fleet size and additional battery under different instances.

8. Discussion and conclusion

The utilization of SDRs in food delivery represents a promising approach to addressing persistent logistical challenges in the rapidly expanding OFD sector. Despite their potential, existing literature rarely explores the operational intricacies of SDR systems under stochastic demand conditions, particularly concerning integrated battery-swapping strategies. This research contributes to the literature by developing a two-stage stochastic optimization model specifically designed for a single-depot SDR system, integrating essential operational elements such as battery swapping and penalty-based soft time windows. The primary goal is to determine the optimal fleet size and the required number of additional batteries to minimize the expected total system cost effectively.

We propose an innovative solution methodology combining CA models and heuristic techniques to efficiently address the complexity inherent in stochastic routing problems. By strategically solving a limited set of scenarios and fitting these results using a tailored approximation model, our method accurately estimates the expected system cost across various resource allocation decisions without exhaustive scenario evaluations. This approach substantially reduces computational efforts while maintaining

solution quality and practical applicability. To further assess solution robustness, extended validation processes involving additional scenario simulations are implemented.

In solving the second-stage routing problems, we employ an ALNS-based heuristic, specifically customized to handle PDPTW, augmented with battery swapping decisions and late-order penalties. The effectiveness of this heuristic has been validated through rigorous numerical experiments, demonstrating its superiority in generating high-quality solutions within significantly shorter computational time.

However, several limitations exist in the current study. Firstly, the developed model does not incorporate recourse actions explicitly, limiting its adaptability in dynamically evolving operational contexts. Potential service failures resulting from resource inadequacy need further exploration in future extensions of the model. Moreover, while our validation has proven the model's effectiveness in controlled scenarios, future research should involve comprehensive validation using real-world operational data, potentially enhancing the model's reliability and further expand the applicable scenarios.

From an applied perspective, the proposed model is designed for sidewalk delivery robots but maintains a structure that is generalizable from classical EV-PDPTW formulations. As such, it can be readily extended to other electric delivery platforms, such as electric vans or cargo bikes, through appropriate parameter settings and minor constraint adjustments. Future investigations could explore comparative analyses of SDR efficiency against these alternative delivery modes (e.g. cargo bikes) under varied cost and infrastructure scenarios, leveraging the same modeling framework. Additionally, examining the integration of SDRs within broader multimodal transportation systems presents another promising direction. Cooperative models that coordinate between diverse transport modes could better exploit each mode's strengths, ultimately improving overall system efficiency and sustainability.

CRediT authorship contribution statement

Yuchen Du: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Hai Yang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Joseph Y.J. Chow:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Tho V. Le:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

On behalf of all authors, the corresponding author states that there is no conflict of interest.

Acknowledgment

This work was supported by the National Science Foundation (NSF) grants CMMI-2423908 and CMMI-2423909.

Data availability

Data will be made available on request.

References

Ai, T.J., Kachitvichyanukul, V., 2009. A particle swarm optimization for the vehicle routing problem with simultaneous pickup and delivery. Comput. Oper. Res. 36 (5), 1693–1702.

Al-Kanj, L., Nascimento, J., Powell, W.B., 2020. Approximate dynamic programming for planning a ride-hailing system using autonomous fleets of electric vehicles. European J. Oper. Res. 284 (3), 1088–1106.

Alvarez, A., Cordeau, J.-F., Jans, R., 2024. The consistent vehicle routing problem with stochastic customers and demands. Transp. Res. Part B: Methodol. 186, 102968.

Alverhed, E., Hellgren, S., Isaksson, H., Olsson, L., Palmqvist, H., Flodén, J., 2024. Autonomous last-mile delivery robots: A literature review. Eur. Transp. Res. Rev. 16 (1), 4.

Aziez, I., Côté, J.-F., Coelho, L.C., 2020. Exact algorithms for the multi-pickup and delivery problem with time windows. European J. Oper. Res. 284 (3), 906–919.

Baldacci, R., Bartolini, E., Mingozzi, A., 2011. An exact algorithm for the pickup and delivery problem with time windows. Oper. Res. 59 (2), 414–426.

Bergmann, F.M., Wagner, S.M., Winkenbach, M., 2020. Integrating first-mile pickup and last-mile delivery on shared vehicle routes for efficient urban e-commerce distribution. Transp. Res. Part B: Methodol. 131, 26–62.

Boeing, G., 2017. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. URL https://osmnx.readthedocs.io/en/stable/. (Accessed 24 July 2024).

Bongiovanni, C., Kaspi, M., Geroliminis, N., 2019. The electric autonomous dial-a-ride problem. Transp. Res. Part B: Methodol. 122, 436–456.

Chowbus, 2024. https://www.chowbus.com/. (Accessed 29 July 2024).

Christiaens, J., Vanden Berghe, G., 2020. Slack induction by string removals for vehicle routing problems. Transp. Sci. 54 (2), 417–433.

DoorDash, 2024. https://www.doordash.com/. (Accessed 29 July 2024).

Erdelić, T., Carić, T., 2019. A survey on the electric vehicle routing problem: Variants and solution approaches. J. Adv. Transp. 2019 (1), 5075671.

Erdelić, T., Carić, T., 2022. Goods delivery with electric vehicles: Electric vehicle routing optimization with time windows and partial or full recharge. Energies 15 (1), 285.

Figliozzi, M.A., 2009. Planning approximations to the average length of vehicle routing problems with time window constraints. Transp. Res. Part B: Methodol. 43 (4), 438–447.
Garus, A., Christidis, P., Mourtzouchou, A., Duboz, L., Ciuffo, B., 2024. Unravelling the last-mile conundrum: A comparative study of autonomous delivery robots, delivery bicycles, and light commercial vehicles in 14 varied European landscapes. Sustain. Cities Soc. 108, 105490.
Ghilas, V., Demir, E., Van Woensel, T., 2016. A scenario-based planning for the pickup and delivery problem with time windows, scheduled lines and stochastic demands. Transp. Res. Part B: Methodol. 91, 34–51.
Goeke, D., 2019. Granular tabu search for the pickup and delivery problem with time windows and electric vehicles. European J. Oper. Res. 278 (3), 821–836.
Grandinetti, L., Guerriero, F., Pezzella, F., Pisacane, O., 2016. A pick-up and delivery problem with time windows by electric vehicles. Int. J. Prod. Qual. Manag. 18 (2–3), 403–423.
Hagberg, A.A., Schult, D.A., Swart, P.J., 2024. NetworkX. URL https://networkx.org/. (Accessed 24 July 2024).
Hiermann, G., Puchinger, J., Ropke, S., Hartl, R.F., 2016. The electric fleet size and mix vehicle routing problem with time windows and recharging stations. European J. Oper. Res. 252 (3), 995–1018.
Jennings, D., Figliozzi, M., 2019. Study of sidewalk autonomous delivery robots and their potential impacts on freight efficiency and travel. Transp. Res. Rec. 2673 (6), 317–326.
Jie, W., Yang, J., Zhang, M., Huang, Y., 2019. The two-echelon capacitated electric vehicle routing problem with battery swapping stations: Formulation and efficient methodology. European J. Oper. Res. 272 (3), 879–904.
Keskin, M., Çatay, B., 2016. Partial recharge strategies for the electric vehicle routing problem with time windows. Transp. Res. Part C: Emerg. Technol. 65, 111–127.
Kiwibot, 2024. https://www.kiwibot.com/. (Accessed 17 July 2024).
Kleywegt, A.J., Shapiro, A., Homem-de Mello, T., 2002. The sample average approximation method for stochastic discrete optimization. SIAM J. Optim. 12 (2), 479–502.
Ma, T.-Y., Fang, Y., Connors, R.D., Viti, F., Nakao, H., 2024. A hybrid metaheuristic to optimize electric first-mile feeder services with charging synchronization constraints and customer rejections. Transp. Res. Part E: Logist. Transp. Rev. 185, 103505.
Mourad, A., Puchinger, J., Van Woensel, T., 2021. Integrating autonomous delivery service into a passenger transportation system. Int. J. Prod. Res. 59 (7), 2116–2139.
Nanry, W.P., Barnes, J.W., 2000. Solving the pickup and delivery problem with time windows using reactive tabu search. Transp. Res. Part B: Methodol. 34 (2), 107–121.
Pantelidis, T.P., Chow, J.Y., Cats, O., 2024. Mobility operator service capacity sharing contract design to risk-pool against network disruptions. Transp. A: Transp. Sci. 20 (3), 2210229.
Raeesi, R., Zografos, K.G., 2020. The electric vehicle routing problem with time windows and synchronised mobile battery swapping. Transp. Res. Part B: Methodol. 140, 101–129.
Ropke, S., Cordeau, J.-F., Laporte, G., 2007. Models and branch-and-cut algorithms for pickup and delivery problems with time windows. Networks: An Int. J. 49 (4), 258–272.
Ropke, S., Pisinger, D., 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp. Sci. 40 (4), 455–472.
Savelsbergh, M.W., Sol, M., 1995. The general pickup and delivery problem. Transp. Sci. 29 (1), 17–29.
Schneider, M., Stenger, A., Goeke, D., 2014. The electric vehicle-routing problem with time windows and recharging stations. Transp. Sci. 48 (4), 500–520.
Shaw, P., 1997. A New Local Search Algorithm Providing High Quality Solutions to Vehicle Routing Problems 46. APES Group, Dept of Computer Science, University of Strathclyde, Scotland, UK.
Solomon, M.M., 1987. Algorithms for the vehicle routing and scheduling problems with time window constraints. Oper. Res. 35 (2), 254–265.
Starship Technologies, 2024. https://www.starship.xyz/. (Accessed 17 July 2024).
Uber Eats, 2024. https://www.ubereats.com/. (Accessed 29 July 2024).
Van Slyke, R.M., Wets, R., 1969. L-shaped linear programs with applications to optimal control and stochastic programming. SIAM J. Appl. Math. 17 (4), 638–663.
Wang, Z., Dessouky, M., Van Woensel, T., Ioannou, P., 2023. Pickup and delivery problem with hard time windows considering stochastic and time-dependent travel times. EURO J. Transp. Logist. 12, 100099.
Wang, C., Mu, D., Zhao, F., Sutherland, J.W., 2015. A parallel simulated annealing method for the vehicle routing problem with simultaneous pickup–delivery and time windows. Comput. Ind. Eng. 83, 111–122.
Wevolver, 2024. Starship technologies - starship robot. URL https://www.wevolver.com/specs/starship-technologies-starship-robot. (Accessed 11 November 2024).
Wiki, 2024. Purdue memorial union - Wikipedia. URL https://en.wikipedia.org/wiki/Purdue_Memorial_Union. (Accessed 24 July 2024).
Yang, H., Du, Y., Le, T., Chow, J.Y., 2025. Analytical model for large-scale design of sidewalk delivery robot systems with bounded makespans. Transp. Res. Part C: Emerg. Technol. 171, 104978.
Yang, J., Sun, H., 2015. Battery swap station location-routing problem with capacitated electric vehicles. Comput. Oper. Res. 55, 217–232.